SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

NAME: SARANYA.R.A

REG NO: 192572052

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

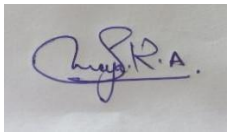
Annexure A

Scenario Based Assignment

Code of Conduct:

I Saranya.R.A(192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO2 – ASSESSMENT TOOL 2

Scenario Based Assignment

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

QUESTIONS: 5 **Duration:** 1 Hour **Total Marks:** 50

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total (50)
Understanding of Scenario	20% (2)	/2	/2	/2	/2	/2	/10
Identification of Key Issues	20% (2)	/2	/2	/2	/2	/2	/10
Application of Concepts	25% (2.5)	/2.5	/2.5	/2.5	/2.5	/2.5	/12.5
Decision Making	20% (2)	/2	/2	/2	/2	/2	/10
Clarity of Response	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/50

100

1. AI-Powered Drone for Emergency Medicine Delivery (Greedy Best-First Search & A*)

Introduction

An AI-powered drone must deliver emergency medicines in a flood-affected region. The environment changes frequently due to blocked roads, weather, and varying travel costs. The search algorithm should find a safe and fast route.

i. Greedy Best-First Search (GBFS)

Working

Greedy Best-First Search selects the node with the smallest heuristic value $h(n)$, usually the straight-line distance to the destination.

$$f(n) = h(n)$$

The algorithm ignores the actual travel cost and chooses the path that appears closest to the goal.

Strengths

- Very fast.
- Low computation time.
- Suitable when an immediate decision is required.
- Can quickly reach the destination if the heuristic is accurate.

Weaknesses

- Does not consider travel cost.
- May choose risky or blocked routes.
- Not guaranteed to find the shortest or safest path.
- May get trapped in dead ends.

Example

A flooded road may appear closer to the destination but be completely inaccessible. GBFS may repeatedly attempt this path.

*ii. A Search**

Working

A* combines the actual cost and heuristic estimate.

$$f(n) = g(n) + h(n)$$

Where

- g(n) = Cost travelled so far
- h(n) = Estimated remaining distance

Example:

Route	g(n)	h(n)	f(n)
Route A	20	15	35
Route B	12	28	40

The algorithm chooses the path with the smallest f(n).

Advantages

- Finds the optimal path (with an admissible heuristic).
- Considers weather and terrain costs.
- Avoids unnecessarily expensive routes.
- Produces safer decisions.

Disadvantages

- Requires more memory.
- Slightly slower than GBFS.

iii. Impact of Heuristic Accuracy

Greedy Best-First Search

- Highly dependent on heuristic accuracy.
- Good heuristic → Fast solution.
- Poor heuristic → Wrong decisions and unnecessary exploration.

A*

- Accurate heuristic → Faster search.
- Less accurate heuristic → Still finds the optimal path but explores more nodes.

Heuristic Accuracy	GBFS	A*
High	Excellent	Excellent

Heuristic Accuracy	GBFS	A*
Medium	Moderate	Good
Poor	Poor	Still Optimal but Slower

iv. Effect of Dynamic Changes (Blocked Paths)
When roads suddenly become blocked:
Greedy Best-First Search

- May continue toward blocked routes.
- Requires restarting the search.
- Performance decreases significantly.

A*

- Updates path cost.
- Recalculates better alternatives.
- More reliable in changing environments.

v. Recommended Algorithm
Recommendation: A*
Justification

Criteria	Greedy BFS	A*
Speed	Excellent	Very Good
Safety	Low	High
Handles Variable Cost	No	Yes
Optimal Route	No	Yes
Dynamic Environment	Moderate	Excellent

Conclusion
A* is the best choice because it balances speed, optimality, and safety. It can adapt to changing terrain, weather, and blocked paths while minimizing travel delay.

2. Smart City Traffic Signal Optimization
Problem Formulation
Objective Function
Minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Decision Variables

- Green signal duration
- Signal sequence
- Signal offsets

Constraints

- Traffic flow changes continuously.
- Hundreds of intersections.
- Pedestrian crossings.
- Emergency vehicle priority.

Why Traditional Search is Inefficient

Traditional search explores every possible signal timing combination.
For hundreds of intersections:

$$\text{Search Space} \approx b^n$$

where:

- b = Possible timings
- n = Number of intersections

The search space becomes enormous, making exhaustive search impractical.

i. Hill Climbing

Working

Hill Climbing repeatedly improves the current solution by making small adjustments.
Example:

- Increase green light by 5 seconds.
- Measure congestion.
- Keep the change only if it improves traffic.

Advantages

- Simple.
- Fast.
- Requires little memory.

Limitations

- Can become stuck in local optima.
- Cannot backtrack.
- Sensitive to the starting solution.

ii. Local Maxima, Plateaus, and Ridges

Local Maximum

A signal timing appears optimal locally but is not the best overall.
Example: One intersection improves while the entire city's traffic worsens.

Plateau

Many neighboring solutions have identical quality.
Example: Changing green time by 1 second produces no noticeable improvement.

Ridge

Improvement requires coordinated changes across multiple intersections.
Example: Adjusting one signal alone has little effect, but synchronizing several signals significantly reduces congestion.

iii. Simulated Annealing & Genetic Algorithms

Simulated Annealing

Works like Hill Climbing but occasionally accepts worse solutions.
Advantages:

- Escapes local maxima.
- Better exploration.
- Suitable for dynamic optimization.

Genetic Algorithm (GA)
Works using:

- Population
- Selection
- Crossover
- Mutation

Advantages:

- Explores many solutions simultaneously.
- Handles very large search spaces.
- Adapts to changing traffic conditions.
- Finds near-optimal solutions.

iv. Recommended Approach
Recommendation: Genetic Algorithm
Justification

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Scalability	Low	Medium	High
Escapes Local Maxima	No	Yes	Yes
Dynamic Adaptation	Poor	Good	Excellent
Large Search Space	Poor	Good	Excellent

Conclusion
A Genetic Algorithm is the most suitable because it is highly scalable, adapts to changing traffic conditions, and effectively searches large solution spaces.

3. Mars Autonomous Rover (Online Search Agent)
i. Why Online Search?
The rover has:

- No complete map.
- Limited sensing.
- Unknown terrain.
- Must make decisions while exploring.

Offline search requires a complete map beforehand, which is unavailable. Therefore, an online search agent is necessary.

ii. Challenges
Partial Observability

- Only nearby terrain is visible.
- Hidden obstacles remain unknown until approached.

Dynamic Environment

- Dust storms.
- Loose rocks.
- Changing terrain conditions.

These uncertainties require continuous adaptation.

iii. Internal Model
The rover builds an internal map by:

1. Sensing nearby terrain.
2. Recording safe and hazardous areas.
3. Updating obstacle locations.
4. Planning future movements using accumulated knowledge.

This evolving map improves navigation over time.

iv. Online vs. Uninformed vs. Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Complete Map Needed	No	Yes	Yes
Learns While Moving	Yes	No	No
Adapts to Changes	Yes	Limited	Limited
Suitable for Mars	Yes	No	Moderate

v. Recommended Approach

Recommendation: Online Search Agent

Justification

- Works without prior knowledge.
- Continuously updates its map.
- Avoids hazards.
- Adapts to uncertainty.
- Maximizes mission safety and exploration efficiency.

4. University Exam Timetabling (Constraint Satisfaction Problem)

Problem Formulation

Variables

Each exam/course is a variable.

Example:

- E1 = AI
- E2 = DBMS
- E3 = Networks

Domains

Possible exam slots.

Example:

- Monday Morning
- Monday Afternoon
- Tuesday Morning

Constraints

- No student has overlapping exams.
- Limited exam halls.
- Subject ordering constraints.
- Faculty availability.
- Special accommodations for students.

ii. Backtracking Search

Working

1. Assign a time slot to an exam.

2. Check constraints.
3. If valid, continue.
4. If a conflict occurs, backtrack and try another slot.

This process continues until all exams are scheduled.

iii. Forward Checking

Forward Checking removes invalid future assignments after each decision.

Example:

- AI is scheduled Monday Morning.
- Any conflicting courses cannot use that slot.

Benefits

- Detects conflicts early.
- Reduces unnecessary exploration.
- Speeds up timetable generation.

iv. Real-World Challenges

- Thousands of students.
- Hundreds of courses.
- Limited resources.
- Last-minute changes.

Best Strategy

Backtracking + Forward Checking + Constraint Propagation

This combination efficiently handles complex constraints and scales well for large universities.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI tries to maximize its score.
- Opponent tries to minimize the AI's score.

The algorithm explores possible moves and chooses the one with the best worst-case outcome.

ii. Computational Challenges

If:

- Branching factor = b
- Depth = d

Then:

$$\text{Time Complexity} = O(b^d)$$

Large games produce enormous search trees, making exhaustive evaluation impractical within real-time limits.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning skips branches that cannot influence the final decision.

Advantages

- Reduces node evaluations.
- Produces the same optimal move as Minimax.
- Enables deeper searches within the same time limit.

Example

If one move already guarantees a better outcome, inferior branches are ignored without

evaluation.

iv. Evaluation Function

A suitable evaluation function for a real-time strategy game can be:

$$\text{Score} = (5 \times \text{Resources}) + (4 \times \text{Army Strength}) + (3 \times \text{Map Control}) + (2 \times \text{Defense}) - (5 \times \text{Enemy Strength})$$

Factors considered:

- Available resources.
- Military power.
- Territory control.
- Defensive structures.
- Opponent's strength.

These factors reflect both offensive and defensive capabilities.

v. Recommended Strategy

Recommendation: Minimax with Alpha-Beta Pruning

Justification

Criteria	Minimax	Alpha-Beta Pruning
Optimal Decision	Yes	Yes
Search Speed	Slow	Faster
Real-Time Performance	Limited	Good
Large Game Trees	Poor	Excellent

Conclusion

The best approach is Minimax enhanced with Alpha-Beta Pruning. It maintains optimal decision-making while significantly reducing the number of explored game states, making it suitable for real-time strategy games with large decision trees.